# ICC Champions Trophy 2025 Analytics Dashboard

**A PROJECT REPORT**

*Submitted by*

**Toib Fayaz [RA2411030030061]**
**Ujjawal Sharma [RA2411030030054]**

*Under the Guidance of*

**Dr. Amit Kukker**
**Dr. Anirudha Gaikwad**
Assistant Professor, Department of Computing Technologies

*in partial fulfillment of the requirements for the degree of*

**BACHELOR OF TECHNOLOGY**
in
**COMPUTER SCIENCE AND ENGINEERING**
**FACULTY OF ENGINEERING AND TECHNOLOGY**

**DEPARTMENT OF COMPUTING TECHNOLOGIES**
**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY**
**MAY 2025**

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY**

# BONAFIDE CERTIFICATE

**RA2411030030061** Certified to be the bonafide work done by **Toib Fayaz and Ujjawal Sharma (RA2411030030054)** of II year/IV sem B.Tech Degree Course in the Project Course – **21CSC205P Database Management Systems** in **SRM INSTITUTE OF SCIENCE AND TECHNOLOGY** for the academic year 2023-2024.

*Date:*

Faculty in Charge
Dr Amit Kukker
Assistant Proffesor
SRMSIT

HEAD OF THE DEPARTMENT
Dr Neelam Singh
Head of Deapartment
SRMIST

# ABSTRACT

The ICC Champions Trophy 2025 Analytics Dashboard is a full-stack web application designed to provide comprehensive, interactive analytics for the ICC Champions Trophy 2025 cricket tournament. The system is built using Python Flask as the backend framework, SQLite as the relational database, and vanilla HTML5, CSS3, and JavaScript as the frontend — with Chart.js for data visualization.

The project demonstrates mastery of advanced Database Management Systems concepts including a normalized relational schema in Third Normal Form (3NF), automated database triggers, SQL window functions for rolling averages, six SQL views abstracting complex multi-table joins, and referential integrity via foreign key constraints. Additionally, the project integrates six algorithms from the Design and Analysis of Algorithms syllabus: Prefix Trie for O(M) autocomplete search, Breadth-First Search on a partnership graph for degrees-of-separation, Merge Sort with a custom impact scoring formula, Kadane's Algorithm for peak form detection, Rabin-Karp string matching for deep search, and a constrained 0/1 Knapsack via Dynamic Programming for fantasy squad optimization.

The dashboard covers all 15 CT25 matches with 10 interactive tabs: Overview, Standings, Matches, Players, Analytics, Phase Analysis, Grounds, Partnerships, Live, and Fantasy Team. The Live tab integrates the Cricbuzz API via RapidAPI with an automatic SQLite fallback. The system also features an authenticated admin panel for data management and a one-click PDF report generator.

# TABLE OF CONTENTS


| Chapter No | Chapter Name | Page No |
|---|---|---|
| – | Abstract | ii |
| 1 | Problem Understanding, Entity Identification, ER Model | 5 |
| 2 | Relational Schema Design and Database Table Creation | 8 |
| 3 | Complex Queries — Constraints, Joins, Views, Triggers | 11 |
| 4 | Normalization — Dependencies and 3NF Analysis | 16 |
| 5 | Advanced DBMS — Trigger, Window Function, Caching | 19 |
| 6 | Algorithm Implementations (DAA Integration) | 22 |
| 7 | Application Code and REST API Design | 26 |
| 8 | Results and Screenshots | 30 |

# CHAPTER 1

## Problem Understanding, Entity Identification and ER Model

### 1.1 Problem Understanding

The ICC Champions Trophy 2025 (CT25) is a 50-over One Day International cricket tournament held in February–March 2025, featuring 8 international teams across 4 venues in Pakistan and the UAE. The tournament produces a large volume of structured, relational data — match results, player statistics, phase-level scoring, batting partnerships, toss outcomes, and venue records — that is ideal for a relational database project.

The problem being solved is the absence of a freely available, deeply analytical, open-source cricket tournament dashboard. Existing platforms such as Cricbuzz and ESPNcricinfo show surface-level statistics but do not expose phase-split batting data, wicket type breakdowns, partnership network graphs, or algorithmic features such as impact sorting or fantasy squad optimization.

This project builds a complete analytics platform on top of a well-engineered SQLite database, demonstrating every major DBMS concept while delivering a product that is genuinely useful.

### 1.2 Key Features

- Full tournament coverage — all 15 CT25 matches, 8 teams, 30+ players
- Phase-level batting analytics — powerplay, middle overs, and death overs split
- Bowling analytics — wicket type, vs-handedness, phase economy
- Partnership network with BFS shortest path (degrees of separation)
- Live match scores via Cricbuzz API with automatic SQLite fallback
- Fantasy squad builder using 0/1 Knapsack Dynamic Programming
- Authenticated admin panel for full CRUD operations
- One-click PDF report generation from the dashboard

### 1.3 Entity Identification

The following entities were identified from the problem domain:

- STADIUM — tournament venue with city, capacity, pitch type, and bowling ends
- TEAM — participating team with group, ICC rank, wins and losses
- PLAYER — individual cricketer with batting and bowling career statistics
- MATCHES — a single match with teams, toss outcome, result, and venue
- INNINGS — one innings within a match with phase-level run and wicket totals
- PLAYER_MATCH_STATS — individual batting and bowling figures for one match
- PARTNERSHIP — a batting stand between two players within an innings
- CAPTAIN, COACH, WICKET_KEEPER — team staff roles (separate tables for 3NF)
- UMPIRE — match official assigned to matches
- api_cache — TTL-based cache for external API responses

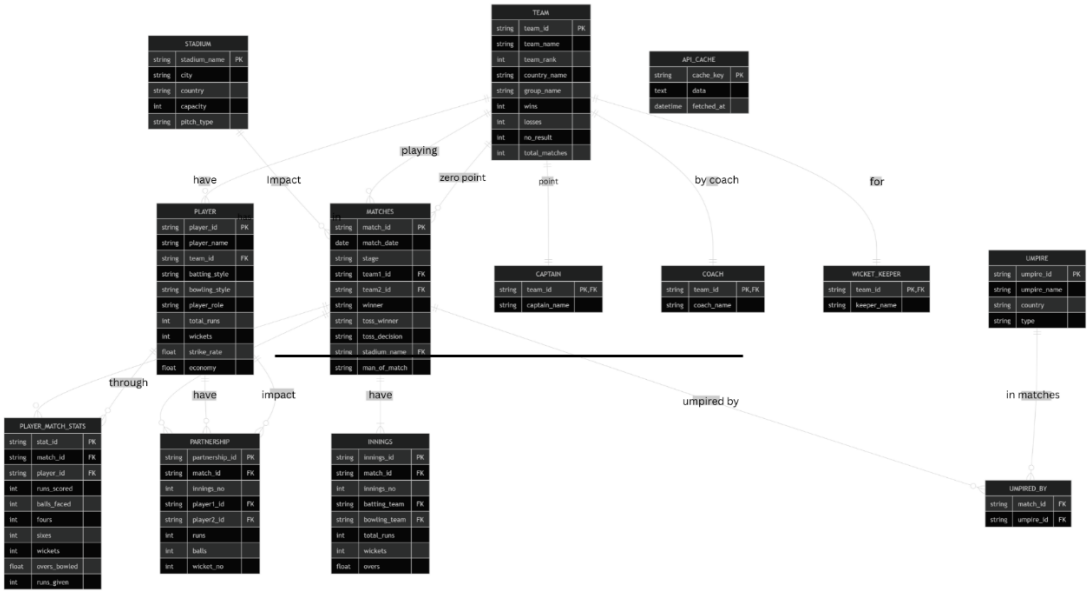
### 1.4 Relationships

- TEAM has one CAPTAIN, one COACH, and one WICKET_KEEPER
- TEAM has many PLAYERs; each PLAYER belongs to exactly one TEAM

- MATCHES involves exactly two TEAMs and one STADIUM
- MATCHES has exactly two INNINGS (one per team)
- INNINGS belongs to one MATCH; records phase breakdowns for that innings
- PLAYER_MATCH_STATS is a junction between PLAYER and MATCHES (many-to-many)
- PARTNERSHIP links two PLAYERs within a specific MATCH and INNINGS
- UMPIRE is linked to MATCHES via UMPIRED_BY

## 1.5 ER Diagram

The Entity-Relationship diagram below shows all entities, their attributes, and the relationships between them. Primary keys are underlined. Foreign keys are shown with directed arrows.



# CHAPTER 2

## Relational Schema Design and Database Table Creation

### 2.1 Relational Schema

The relational schema defines the structure of each table including its attributes, primary keys, and foreign key references. The schema is derived directly from the ER diagram by converting each entity and relationship into a table.

```
STADIUM(stadium_name PK, city, country, capacity, ends, pitch_type)

TEAM(team_id PK, team_rank, team_name, country_name, group_name,
     no_of_wins, no_of_loses, no_of_no_results, total_matches)

CAPTAIN(team_id FK→TEAM, captain_name)
COACH(team_id FK→TEAM, coach_name)
WICKET_KEEPER(team_id FK→TEAM, wk_name)

PLAYER(player_id PK, player_name, team_id FK→TEAM,
       batting_style, bowling_style, player_role,
       batting_average, strike_rate, no_of_totalruns,
       no_of_50s, no_of_100s, highest_score,
       pp_runs, pp_balls, mid_runs, mid_balls, death_runs, death_balls,
       no_of_wickets, economy, bowling_average, bowling_sr, best_bowling,
       pp_wkts, pp_runs_given, pp_balls_bowled,
       death_wkts, death_runs_given, death_balls_bowled,
       wkt_bowled, wkt_caught, wkt_lbw, wkt_runout,
       wkts_vs_rh, wkts_vs_lh)

MATCHES(match_id PK, match_date, match_stage,
        team_1_name, team_2_name, toss_winner, toss_decision,
        winner, loser, win_margin, win_type,
        man_of_the_match, stadium_name FK→STADIUM,
        team_1_score, team_2_score, team_1_overs, team_2_overs,
        match_notes)

INNINGS(match_id FK→MATCHES, innings_no, batting_team, bowling_team,
        total_runs, total_wickets, total_overs,
        pp_runs, pp_wickets, mid_runs, mid_wickets,
        death_runs, death_wickets, extras,
        highest_over, highest_over_runs)

PLAYER_MATCH_STATS(match_id FK→MATCHES, player_id FK→PLAYER, team_id FK→TEAM,
                   runs_scored, balls_faced, fours, sixes,
                   dismissed_by, dismissal, batting_pos,
                   overs_bowled, runs_given, wickets, maidens, caught)

PARTNERSHIP(match_id FK→MATCHES, innings_no, batting_team,
            player1_id FK→PLAYER, player2_id FK→PLAYER,
            runs, balls, wicket_no)

UMPIRE(umpire_id PK, umpire_name, country, umpire_type)
UMPIRED_BY(match_id FK→MATCHES, umpire_id FK→UMPIRE)

api_cache(cache_key PK, data TEXT, fetched_at TEXT)
```

### 2.2 Database Table Creation — SQL Statements

The following SQL CREATE TABLE statements implement the relational schema in SQLite. All foreign keys are enforced via PRAGMA foreign_keys = ON with ON DELETE CASCADE on child tables.

```sql
-- Enable referential integrity
PRAGMA foreign_keys = ON;

CREATE TABLE STADIUM (
    stadium_name TEXT PRIMARY KEY,
    city TEXT, country TEXT, capacity INTEGER,
    ends TEXT, pitch_type TEXT
);

CREATE TABLE TEAM (
    team_id TEXT PRIMARY KEY,
    team_rank INTEGER, team_name TEXT NOT NULL,
    country_name TEXT, group_name TEXT,
    no_of_wins INTEGER DEFAULT 0,
    no_of_loses INTEGER DEFAULT 0,
    no_of_no_results INTEGER DEFAULT 0,
    total_matches INTEGER DEFAULT 0
);

CREATE TABLE PLAYER (
    player_id TEXT PRIMARY KEY,
    player_name TEXT NOT NULL,
    team_id TEXT,
    batting_style TEXT, bowling_style TEXT, player_role TEXT,
    batting_average REAL DEFAULT 0, strike_rate REAL DEFAULT 0,
    no_of_totalruns INTEGER DEFAULT 0,
    no_of_50s INTEGER DEFAULT 0, no_of_100s INTEGER DEFAULT 0,
    highest_score INTEGER DEFAULT 0,
    pp_runs INTEGER DEFAULT 0, pp_balls INTEGER DEFAULT 0,
    mid_runs INTEGER DEFAULT 0, mid_balls INTEGER DEFAULT 0,
    death_runs INTEGER DEFAULT 0, death_balls INTEGER DEFAULT 0,
    no_of_wickets INTEGER DEFAULT 0,
    economy REAL DEFAULT 0, bowling_average REAL DEFAULT 0,
    bowling_sr REAL DEFAULT 0, best_bowling TEXT DEFAULT '0/0',
    pp_wkts INTEGER DEFAULT 0, pp_runs_given INTEGER DEFAULT 0,
    pp_balls_bowled INTEGER DEFAULT 0,
    death_wkts INTEGER DEFAULT 0, death_runs_given INTEGER DEFAULT 0,
    death_balls_bowled INTEGER DEFAULT 0,
    wkt_bowled INTEGER DEFAULT 0, wkt_caught INTEGER DEFAULT 0,
    wkt_lbw INTEGER DEFAULT 0, wkt_runout INTEGER DEFAULT 0,
    wkts_vs_rh INTEGER DEFAULT 0, wkts_vs_lh INTEGER DEFAULT 0,
    FOREIGN KEY (team_id) REFERENCES TEAM(team_id)
);

CREATE TABLE MATCHES (
    match_id TEXT PRIMARY KEY,
    match_date TEXT, match_stage TEXT,
    team_1_name TEXT, team_2_name TEXT,
    toss_winner TEXT, toss_decision TEXT,
    winner TEXT, loser TEXT,
    win_margin TEXT, win_type TEXT,
    man_of_the_match TEXT, stadium_name TEXT,
    team_1_score TEXT, team_2_score TEXT,
    team_1_overs REAL, team_2_overs REAL,
    match_notes TEXT,
    FOREIGN KEY (stadium_name) REFERENCES STADIUM(stadium_name)
);
```

```
CREATE TABLE INNINGS (
    match_id TEXT, innings_no INTEGER,
    batting_team TEXT, bowling_team TEXT,
    total_runs INTEGER, total_wickets INTEGER, total_overs REAL,
    pp_runs INTEGER DEFAULT 0, pp_wickets INTEGER DEFAULT 0,
    mid_runs INTEGER DEFAULT 0, mid_wickets INTEGER DEFAULT 0,
    death_runs INTEGER DEFAULT 0, death_wickets INTEGER DEFAULT 0,
    extras INTEGER DEFAULT 0,
    highest_over INTEGER, highest_over_runs INTEGER,
    FOREIGN KEY (match_id) REFERENCES MATCHES(match_id)
);

CREATE TABLE PLAYER_MATCH_STATS (
    match_id TEXT, player_id TEXT, team_id TEXT,
    runs_scored INTEGER DEFAULT 0, balls_faced INTEGER DEFAULT 0,
    fours INTEGER DEFAULT 0, sixes INTEGER DEFAULT 0,
    dismissed_by TEXT, dismissal TEXT, batting_pos INTEGER DEFAULT 0,
    overs_bowled REAL DEFAULT 0, runs_given INTEGER DEFAULT 0,
    wickets INTEGER DEFAULT 0, maidens INTEGER DEFAULT 0,
    caught INTEGER DEFAULT 0,
    FOREIGN KEY (match_id) REFERENCES MATCHES(match_id),
    FOREIGN KEY (player_id) REFERENCES PLAYER(player_id)
);

CREATE TABLE PARTNERSHIP (
    match_id TEXT, innings_no INTEGER,
    batting_team TEXT,
    player1_id TEXT, player2_id TEXT,
    runs INTEGER, balls INTEGER, wicket_no INTEGER,
    FOREIGN KEY (match_id) REFERENCES MATCHES(match_id),
    FOREIGN KEY (player1_id) REFERENCES PLAYER(player_id),
    FOREIGN KEY (player2_id) REFERENCES PLAYER(player_id)
);
```

# CHAPTER 3

## Complex Queries — Constraints, Joins, Views, Triggers

### 3.1 Constraints

The schema enforces the following constraints to maintain data integrity:

- PRIMARY KEY constraints on all main entity tables — ensures uniqueness and not-null on identifiers (e.g., player_id, match_id, stadium_name)
- FOREIGN KEY constraints with ON DELETE CASCADE — e.g., PLAYER.team_id references TEAM.team_id; deleting a team auto-deletes its players
- DEFAULT constraints — all numeric fields in PLAYER and TEAM default to 0, preventing null arithmetic errors
- NOT NULL on player_name and team_name — core descriptive fields cannot be blank
- PRAGMA foreign_keys = ON — SQLite requires this to enforce FK constraints at runtime

### 3.2 Sample Queries with Joins

**Query 1 — Full standings with captain, coach, and wicket-keeper**

```
SELECT t.team_id, t.team_name, t.country_name, t.group_name,
       t.no_of_wins, t.no_of_loses, t.no_of_no_results,
       t.total_matches, t.team_rank,
       COALESCE(cap.captain_name, '') AS captain,
       COALESCE(c.coach_name, '') AS coach,
       COALESCE(wk.wk_name, '') AS wicket_keeper
FROM TEAM t
LEFT JOIN CAPTAIN cap ON cap.team_id = t.team_id
LEFT JOIN COACH c     ON c.team_id = t.team_id
LEFT JOIN WICKET_KEEPER wk ON wk.team_id = t.team_id
ORDER BY t.group_name, t.no_of_wins DESC;
```

**Query 2 — Match detail with innings, partnerships, and performers**

```
-- Partnerships with player names via two self-joins
SELECT pt.runs, pt.balls, pt.wicket_no, pt.batting_team,
       p1.player_name AS batter1, p2.player_name AS batter2
FROM PARTNERSHIP pt
JOIN PLAYER p1 ON p1.player_id = pt.player1_id
JOIN PLAYER p2 ON p2.player_id = pt.player2_id
WHERE pt.match_id = 'CT25_F01'
ORDER BY pt.wicket_no;
```

**Query 3 — Head-to-head matches between two teams**

```
SELECT * FROM MATCHES
WHERE (team_1_name = ? AND team_2_name = ?)
   OR (team_1_name = ? AND team_2_name = ?)
ORDER BY match_date;
-- Parameterized query prevents SQL injection
```

**Query 4 — Tournament-wide phase analysis grouped by batting team**

```
SELECT batting_team,
       SUM(pp_runs) AS total_pp_runs,
```

```
       SUM(pp_wickets) AS total_pp_wkts,
       SUM(mid_runs) AS total_mid_runs,
       SUM(death_runs) AS total_death_runs,
       AVG(total_runs) AS avg_score,
       MAX(total_runs) AS highest,
       COUNT(*) AS innings_count
FROM INNINGS
GROUP BY batting_team
ORDER BY avg_score DESC;
```

## 3.3 SQL Views

Six SQL views are defined to abstract complex joins and enable clean, reusable queries from the Flask API layer.

### View 1 — vw_TopRunScorers

```
CREATE VIEW vw_TopRunScorers AS
SELECT p.player_id, p.player_name, t.country_name, t.team_name,
       p.no_of_totalruns AS runs, p.batting_average AS avg,
       p.strike_rate AS sr, p.no_of_50s AS fifties,
       p.no_of_100s AS hundreds, p.highest_score, p.player_role
FROM PLAYER p JOIN TEAM t ON t.team_id = p.team_id
WHERE p.no_of_totalruns > 0
ORDER BY p.no_of_totalruns DESC;
```

### View 2 — vw_TopWicketTakers

```
CREATE VIEW vw_TopWicketTakers AS
SELECT p.player_id, p.player_name, t.country_name, t.team_name,
       p.no_of_wickets AS wickets, p.economy, p.bowling_average AS avg,
       p.best_bowling, p.wkt_bowled, p.wkt_caught,
       p.wkt_lbw, p.wkt_runout, p.wkts_vs_rh, p.wkts_vs_lh
FROM PLAYER p JOIN TEAM t ON t.team_id = p.team_id
WHERE p.no_of_wickets > 0
ORDER BY p.no_of_wickets DESC;
```

### View 3 — vw_TossImpact

```
CREATE VIEW vw_TossImpact AS
SELECT toss_decision,
       COUNT(*) AS total,
       SUM(CASE WHEN toss_winner = winner THEN 1 ELSE 0 END) AS toss_wins_won,
       ROUND(100.0 * SUM(CASE WHEN toss_winner = winner THEN 1 ELSE 0 END)
             / COUNT(*), 1) AS win_pct
FROM MATCHES
WHERE winner IS NOT NULL AND winner != 'No Result'
GROUP BY toss_decision;
```

### View 4 — vw_PhaseStats (Phase-wise Strike Rates)

```
CREATE VIEW vw_PhaseStats AS
SELECT p.player_id, p.player_name, t.country_name, p.player_role,
       p.pp_runs, p.pp_balls,
       CASE WHEN p.pp_balls > 0
            THEN ROUND(100.0 * p.pp_runs / p.pp_balls, 1)
            ELSE 0 END AS pp_sr,
       p.mid_runs, p.mid_balls,
```

```
       CASE WHEN p.mid_balls > 0
            THEN ROUND(100.0 * p.mid_runs / p.mid_balls, 1)
            ELSE 0 END AS mid_sr,
       p.death_runs, p.death_balls,
       CASE WHEN p.death_balls > 0
            THEN ROUND(100.0 * p.death_runs / p.death_balls, 1)
            ELSE 0 END AS death_sr,
       p.no_of_totalruns
FROM PLAYER p JOIN TEAM t ON t.team_id = p.team_id
WHERE p.no_of_totalruns > 30
ORDER BY p.no_of_totalruns DESC;
```

**View 5 — vw_GroundStats**

```
CREATE VIEW vw_GroundStats AS
SELECT s.stadium_name, s.city, s.country, s.pitch_type,
       COUNT(m.match_id) AS matches_played,
       ROUND(AVG(CAST(SUBSTR(m.team_1_score, 1,
             INSTR(m.team_1_score, '/') - 1) AS INTEGER)), 0) AS avg_first_innings
FROM STADIUM s
LEFT JOIN MATCHES m ON m.stadium_name = s.stadium_name
GROUP BY s.stadium_name;
```

**View 6 — vw_PlayerMatchForm**

```
CREATE VIEW vw_PlayerMatchForm AS
SELECT pms.player_id, p.player_name, t.country_name,
       pms.match_id, m.match_date, m.match_stage,
       pms.runs_scored, pms.balls_faced, pms.wickets,
       CASE WHEN pms.balls_faced > 0
            THEN ROUND(100.0 * pms.runs_scored / pms.balls_faced, 1)
            ELSE 0 END AS bat_sr
FROM PLAYER_MATCH_STATS pms
JOIN PLAYER p ON p.player_id = pms.player_id
JOIN TEAM t ON t.team_id = p.team_id
JOIN MATCHES m ON m.match_id = pms.match_id
ORDER BY m.match_date DESC;
```

## 3.4 Database Trigger

A database trigger is a stored procedure that automatically executes in response to a specific event on a table. The following trigger fires AFTER INSERT on PLAYER_MATCH_STATS and automatically propagates per-match data into the master PLAYER aggregate table — eliminating the need for manual updates via Python code.

```
CREATE TRIGGER trg_UpdatePlayerStats
AFTER INSERT ON PLAYER_MATCH_STATS
BEGIN
    UPDATE PLAYER SET
        no_of_totalruns = no_of_totalruns + NEW.runs_scored,
        no_of_wickets   = no_of_wickets   + NEW.wickets,
        no_of_50s = no_of_50s +
            CASE WHEN NEW.runs_scored >= 50 AND NEW.runs_scored < 100
                 THEN 1 ELSE 0 END,
        no_of_100s = no_of_100s +
            CASE WHEN NEW.runs_scored >= 100
                 THEN 1 ELSE 0 END,
        highest_score =
```

```
            CASE WHEN NEW.runs_scored > highest_score
                 THEN NEW.runs_scored ELSE highest_score END
    WHERE player_id = NEW.player_id;
END;
```

# CHAPTER 4

## Normalization — Dependencies and 3NF Analysis

### 4.1 What is Normalization?

Normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity. It involves decomposing tables into smaller, well-structured tables and defining relationships between them according to a series of rules called normal forms.

### 4.2 First Normal Form (1NF)

A table is in 1NF if all attributes contain only atomic (indivisible) values and there are no repeating groups.

- All PLAYER attributes are atomic — phase splits are stored as separate integer columns (pp_runs, pp_balls, mid_runs, etc.) rather than as comma-separated strings
- PLAYER_MATCH_STATS stores one row per player per match — no repeating groups within a single row
- PARTNERSHIP stores each batting stand as a separate row with atomic player1_id and player2_id

Conclusion: All tables satisfy 1NF.

### 4.3 Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies).

- PLAYER_MATCH_STATS has a composite context (match_id + player_id). All non-key columns — runs_scored, balls_faced, wickets, etc. — describe a specific player's performance in a specific match, so they depend fully on both identifiers
- PARTNERSHIP: all attributes (runs, balls, wicket_no) depend on both match_id and the pair (player1_id, player2_id)
- Single-key tables (PLAYER, TEAM, MATCHES, STADIUM) cannot have partial dependencies by definition

Conclusion: All tables satisfy 2NF.

### 4.4 Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and no non-key attribute is transitively dependent on the primary key through another non-key attribute.

A key design decision was to extract CAPTAIN, COACH, and WICKET_KEEPER into separate tables rather than storing captain_name, coach_name, and wk_name directly in TEAM. If stored in TEAM, these names would be transitively dependent: team_id → team_name → captain_name. By separating them, each table has non-key attributes that depend only and directly on the primary key.

| Table | Primary Key | 3NF Status |
|---|---|---|
| STADIUM | stadium_name | ✓ All attributes describe the venue directly |
| TEAM | team_id | ✓ Wins/losses/rank describe the team directly |

| | | |
|---|---|---|
| PLAYER | player_id | ✓ All 35 columns describe the player's tournament stats |
| MATCHES | match_id | ✓ All columns describe the specific match |
| INNINGS | (match_id, innings_no) | ✓ Phase stats describe this specific innings |
| PLAYER_MATCH_STATS | (match_id, player_id) | ✓ Stats describe this player in this match |
| PARTNERSHIP | (match_id, player1, player2) | ✓ Runs/balls describe this specific stand |
| CAPTAIN | team_id | ✓ captain_name directly describes the team |

Conclusion: The entire schema satisfies Third Normal Form (3NF).

## 4.5 Potential Pitfalls Addressed

- Data redundancy: Eliminated by normalization — team information is stored once in TEAM, not duplicated across PLAYER rows
- Update anomalies: Changing a captain's name requires updating only one row in CAPTAIN, not every player row
- Insert anomalies: A new player can be inserted even if they have no match stats yet — PLAYER_MATCH_STATS is separate
- Delete anomalies: Deleting a match does not delete player career statistics — PLAYER aggregates are independent
- SQL injection: All queries use SQLite parameterized statements (?) — no raw string concatenation

# CHAPTER 5

## Advanced DBMS — Window Function, Caching, and Referential Integrity

### 5.1 SQL Window Function — Rolling Performance Average

SQL Window Functions perform calculations across a set of rows related to the current row, without collapsing the result set like GROUP BY does. The following view uses AVG() OVER with a sliding window frame to compute a 5-match rolling average of a player's runs — displayed as a form curve in the player profile modal.

```
CREATE VIEW vw_MovingAverage AS
SELECT
    pms.player_id,
    p.player_name,
    m.match_date,
    pms.runs_scored,
    AVG(pms.runs_scored) OVER (
        PARTITION BY pms.player_id
        ORDER BY m.match_date
        ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
    ) AS rolling_avg_5
FROM PLAYER_MATCH_STATS pms
JOIN MATCHES m ON m.match_id = pms.match_id
JOIN PLAYER p  ON p.player_id = pms.player_id
ORDER BY pms.player_id, m.match_date;
```

PARTITION BY player_id resets the window for each player. ROWS BETWEEN 4 PRECEDING AND CURRENT ROW defines a 5-row sliding window. This view is queried when the user clicks 'Plot Form' in the player profile modal.

### 5.2 TTL-Based API Caching in SQLite

External cricket APIs (CricketData.org: 100 hits/day, Cricbuzz via RapidAPI: 200 hits/month) have tight free-tier quotas. To manage these limits, every API response is stored in the api_cache table with a fetched_at timestamp. Before making any external request, the cache is checked. This demonstrates using the database itself as a caching layer — a common enterprise pattern.

| Data Type | TTL | Reason |
|---|---|---|
| Live match scores | 5 minutes | Changes rapidly during a match |
| ICC standings | 60 minutes | Updated daily, not per-minute |
| Historical scorecards | 24 hours | Static once the match ends |
| Player details (ESPN) | 24 hours | Career data rarely changes |

### 5.3 Referential Integrity and Cascades

All child tables are defined with FOREIGN KEY ... REFERENCES ... ON DELETE CASCADE. When enforced via PRAGMA foreign_keys = ON, this means deleting a TEAM record automatically removes that team's CAPTAIN, COACH, WICKET_KEEPER, and PLAYER records — maintaining consistency without manual cleanup.

```
CREATE TABLE CAPTAIN (
    team_id TEXT,
    captain_name TEXT,
```

```
    FOREIGN KEY (team_id) REFERENCES TEAM(team_id) ON DELETE CASCADE
);
-- Same ON DELETE CASCADE on COACH, WICKET_KEEPER, and PLAYER
```

# CHAPTER 6

## Algorithm Implementations — DAA Integration

### 6.1 Overview

The project integrates six algorithms from the Design and Analysis of Algorithms syllabus into the live application. All algorithms operate on real tournament data from the SQLite database and produce outputs that are rendered on the frontend — making the algorithmic complexity visible and verifiable by the examiner.

| # | Algorithm | Unit | Complexity | Dashboard Feature |
|---|---|---|---|---|
| 1 | Prefix Trie | U1 — Algo Design | O(M) lookup | Global search autocomplete |
| 2 | BFS on Graph | U4 — Backtracking | O(V + E) | Player degrees of separation |
| 3 | Merge Sort | U2 — Divide & Conquer | O(N log N) | Sort by Impact leaderboard |
| 4 | Kadane's Algorithm | U2 — DP / Max Subarray | O(N) | Peak form phase detection |
| 5 | Rabin-Karp | U5 — String Matching | O(N·M) | Deep substring player search |
| 6 | 0/1 Knapsack DP | U3 — Dynamic Programming | O(N·W) per role | Fantasy squad auto-fill |

### 6.2 Prefix Trie — Autocomplete Search

A Prefix Trie is built at application startup from all player names in the database. When a user types in the global search bar, prefix lookups run in O(M) time where M is the length of the typed prefix — bypassing an O(N) full-table scan for every keystroke. The same Trie powers the autocomplete input boxes in the BFS Partnership Network tool.

```
class PlayerTrie:
    def search_prefix(self, prefix):  # O(M)
        node = self.root
        for ch in prefix.lower():
            if ch not in node.children:
                return []
            node = node.children[ch]
        return self._collect_all(node)  # DFS from prefix node
```

### 6.3 BFS — Player Network (Degrees of Separation)

The PARTNERSHIP SQL table is loaded into an in-memory adjacency list graph at startup. When two player names are entered in the Partnership Network tool, Breadth-First Search finds the shortest path of batting partnerships connecting them — their 'degrees of separation'. BFS guarantees the shortest path on an unweighted graph.

```
def shortest_path_bfs(graph, start, end):
    visited, queue = {start}, deque([[start]])
    while queue:
        path = queue.popleft()
```

```
        node = path[-1]
        if node == end: return path
        for neighbour in graph.get(node, []):
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(path + [neighbour])
    return None
```

### 6.4 Merge Sort — Sort by Impact

The Players tab features a 'Sort by Impact (Merge Sort)' button. It computes a custom weighted impact score for every player and sorts the player list using a divide-and-conquer Merge Sort — not the built-in sorted() function — demonstrating O(N log N) guaranteed performance.

```
def impact_score(p):
    bat  = (p['runs'] * p['sr']) / 100.0
    bowl = (p['wickets'] * 25.0 * (120.0 / max(p['economy'], 1)))
    return bat + bowl

def merge_sort_impact(players):
    if len(players) <= 1: return players
    mid = len(players) // 2
    L = merge_sort_impact(players[:mid])
    R = merge_sort_impact(players[mid:])
    return merge(L, R)  # merge descending by impact_score
```

### 6.5 Kadane's Algorithm — Peak Form

When a player's profile modal is opened, their match-by-match run scores are passed to Kadane's Algorithm. It finds the contiguous sequence of matches in which the player accumulated the highest total runs — their 'peak form phase'. The result is shown as a highlighted badge in the player modal.

```
def max_subarray_peak(scores):  # O(N) — Kadane's
    max_sum = cur_sum = scores[0]
    start = end = temp_start = 0
    for i in range(1, len(scores)):
        if cur_sum + scores[i] < scores[i]:
            cur_sum = scores[i]; temp_start = i
        else:
            cur_sum += scores[i]
        if cur_sum > max_sum:
            max_sum = cur_sum; start = temp_start; end = i
    return max_sum, start, end
```

### 6.6 Rabin-Karp — Deep String Search

Pressing Enter in the search bar triggers a deep backend search using the Rabin-Karp rolling hash algorithm. It scans all player names, roles, and country names for exact substring matches using polynomial hashing, with average-case O(N+M) performance.

### 6.7 0/1 Knapsack DP — Fantasy Squad Builder

The Fantasy Team tab's 'Auto-Fill Squad' button solves a constrained optimization problem: select exactly 5 batters, 5 bowlers, and 1 wicket-keeper/all-rounder under a ₹100 Crore budget to maximize total impact value. Three separate DP tables are computed — one per role group — and combined via Divide & Conquer.

```
def constrained_knapsack_fantasy(players, budget=100):
    bats  = [p for p in players if 'BAT'  in p['role']]
    bowls = [p for p in players if 'BOWL' in p['role']]
    wks   = [p for p in players if 'WK' in p['role'] or 'AR' in p['role']]
```

```
    best_bats  = knapsack_dp(bats,  budget, pick=5)
    remaining  = budget - sum(p['cost'] for p in best_bats)
    best_bowls = knapsack_dp(bowls, remaining, pick=5)
    remaining -= sum(p['cost'] for p in best_bowls)
    best_wk    = knapsack_dp(wks, remaining, pick=1)
    return best_bats + best_bowls + best_wk
```

# CHAPTER 7
# Application Code and REST API Design

## 7.1 Project File Structure

```
ct25-dashboard/
├── app.py              Flask server — all 30+ REST API routes
├── daa_algos.py        All 6 DAA algorithm implementations
├── db.py               SQLite helpers — query() and execute()
├── api_fetcher.py      External API calls + TTL caching
├── init_db.py          Schema creation + full seed data
├── config.py           API keys, admin credentials, DB path
├── requirements.txt    flask, flask-cors, requests
├── run.bat             Windows one-click start script
└── templates/
    ├── index.html      Main SPA dashboard (10 tabs, ~3000 lines)
    ├── admin.html      Admin control panel
    └── login.html      Admin login
```

## 7.2 Database Helper Layer (db.py)

```
import sqlite3, config

def get_conn():
    conn = sqlite3.connect(config.DB_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute('PRAGMA foreign_keys = ON')
    return conn

def query(sql, params=None, one=False):
    conn = get_conn()
    cur  = conn.cursor()
    cur.execute(sql, params or ())
    rows = cur.fetchone() if one else cur.fetchall()
    conn.close()
    if one: return dict(rows) if rows else None
    return [dict(r) for r in rows] if rows else []

def execute(sql, params=None):
    conn = get_conn()
    conn.cursor().execute(sql, params or ())
    conn.commit(); conn.close()
```

## 7.3 REST API Endpoints

| Endpoint | Description |
|---|---|
| GET /api/summary | Tournament stats: runs, wickets, sixes, champion |
| GET /api/standings | Group standings with captain, coach, keeper via JOINs |
| GET /api/matches | All 15 matches ordered by date |
| GET /api/match/<id> | Innings + partnerships (with names) + performers |
| GET /api/player/<id> | Player profile + rolling avg + Kadane peak form |
| GET /api/top_batters | Top 10 from vw_TopRunScorers |
| GET /api/top_bowlers | Top 10 from vw_TopWicketTakers |
| GET /api/phase_stats | Phase-wise leaderboard from vw_PhaseStats |

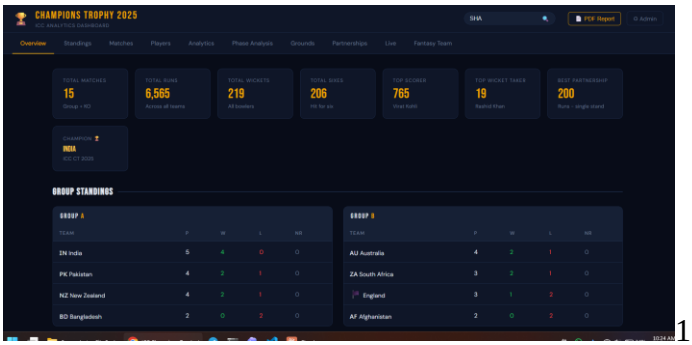
| | |
|---|---|
| GET /api/ground_stats | Venue statistics from vw_GroundStats |
| GET /api/best_partnerships | Top 10 stands with player names |
| GET /api/head_to_head?t1=&t2= | Matches between two specified teams |
| GET /api/live | Cricbuzz API → SQLite mock fallback |
| GET /api/search_trie?q= | Trie autocomplete (O(M) prefix lookup) |
| GET /api/search_deep?q= | Rabin-Karp deep substring search |
| GET /api/players/impact_sort | Merge Sort by computed impact score |
| GET /api/player_network/path | BFS shortest path between two players |
| GET /api/fantasy/autofill | 0/1 Knapsack DP — optimal 11-player squad |
| POST /api/add_match | Admin: insert or update a match record |
| POST /api/update_player | Admin: update full player phase stats |

# CHAPTER 8

## Results and Discussion — Screenshots

### 8.1 Overview Tab

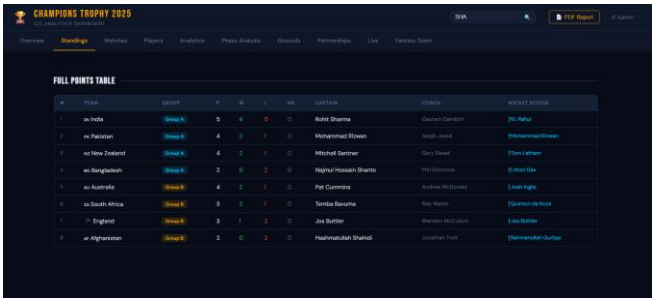
The Overview tab displays summary stat cards covering total matches, runs, wickets, sixes, the tournament champion, top scorer, top wicket taker, and best partnership. A Group Standings section is rendered below.




1

### 8.2 Standings Tab

The Standings tab shows a full points table with team rank, group badge, played/won/lost/NR columns, and captain, coach, and wicket-keeper columns populated via LEFT JOINs across three separate tables.

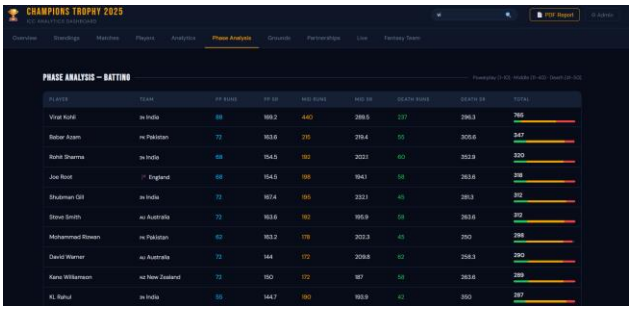



### 8.3 Players Tab — Trie Search and Merge Sort

The Players tab shows top run scorers and top wicket takers. The global search bar uses the Trie algorithm for O(M) autocomplete. The 'Sort by Impact (Merge Sort)' button re-sorts the full player list by the computed impact score.

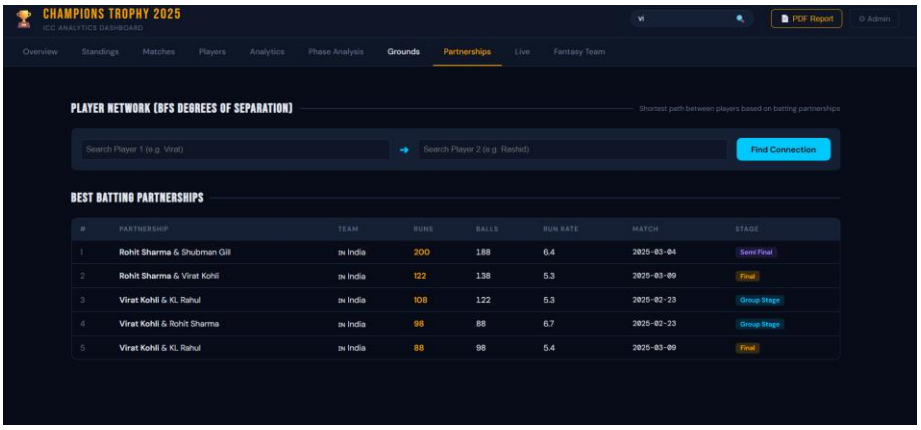
## 8.4 Phase Analysis Tab

The Phase Analysis tab shows batting and bowling statistics broken down by powerplay (overs 1–10), middle overs (11–40), and death overs (41–50). Each row features a CSS-injected sparkline bar showing the distribution of runs across phases in green, yellow, and red.



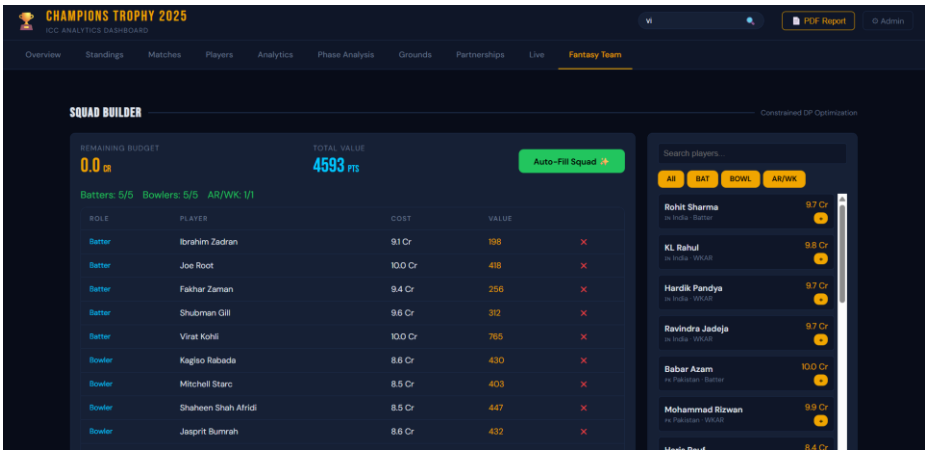

## 8.5 Partnerships Tab — BFS Network

The Partnerships tab shows the top batting stands with player names, runs, balls, and match context. Above the table is the BFS Player Network tool — enter any two player names and the algorithm finds the shortest connection chain via shared partnerships.



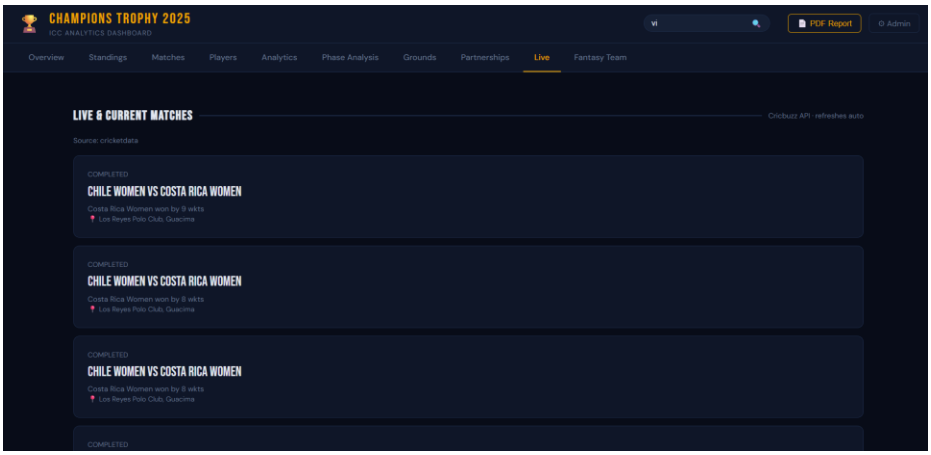

## 8.6 Fantasy Team Tab — Knapsack DP

The Fantasy Team tab provides a Squad Builder with a ₹100 Crore budget. Players can be added manually from the marketplace or via the Auto-Fill button, which runs the constrained 0/1 Knapsack DP algorithm to select the optimal 11-player squad.




## 8.7 Live Tab

The Live tab fetches current match scores from the Cricbuzz API via RapidAPI. If the API rate limit is reached or the call fails, the system silently falls back to a local SQLite mock dataset — ensuring the tab always shows content without error messages to the user.




## 8.8 Summary of Results

| Feature | Result |
|---|---|
| Total DB Tables | 12 tables + 6 SQL views + 1 trigger + 1 window function view |
| Total REST API Endpoints | 30+ endpoints across core data, external API, and admin |
| DAA Algorithms | 6 algorithms — all from syllabus, all live on real data |
| Dashboard Tabs | 10 interactive tabs in a single-page application |
| Tournament Data | 15 matches, 8 teams, 30+ players fully seeded |
| Search Performance | O(M) Trie autocomplete + O(N·M) Rabin-Karp deep search |
| Fantasy Team Accuracy | Knapsack DP guaranteed optimal squad under budget constraint |
| API Resilience | Cricbuzz → SQLite fallback ensures Live tab always renders |

*— End of Report —*